ReSequel: Robust LLM-assisted Query Rewriting and Optimization using Templatization and Sampling

Saeed Fathollahzadeh
Concordia University
saeed.fathollahzadeh@concordia.ca

Essam Mansour
Concordia University
essam.mansour@concordia.ca

Matthias Boehm
Technische Universität Berlin
matthias.boehm@tu-berlin.de

ABSTRACT

Heuristic query rewriting has long complemented cost-based optimization to improve performance. Such rewrites transform SQL queries into semantically equivalent forms that are easier or faster to execute. Examples are standardizing expressions, eliminating redundancy, propagating constants, pushing down selections and projections, unnesting queries, and utilizing constraints. Modern DBMSs implement hundreds to thousands of such rules, but maintaining them is notoriously difficult. The interactions among rules are complex, and their static nature and application order prevent adaptation to specific query and database characteristics. Recent approaches that use large language models (LLMs) for query rewriting show promise but face challenges regarding the large search space, reliable query verification, and exploitation of metadata. We present **ReSequel**, an *outer optimization layer* on top of existing DBMSs to rewrite SQL queries using LLMs. ReSequel leverages catalog and statistical metadata to infer *template-specific rules* that guide the LLM toward effective query transformations. We generate, verify, and rank rewritten query variants on sampled data to ensure result correctness and runtime improvements. Our experiments cover five benchmarks: TPC-H, Stats, Public BI, IMDB, and StackOverflow; multiple DBMSs: PostgreSQL, MySQL, and DuckDB; as well as LLM-based query rewriting baselines. ReSequel achieves up to 3–14× speedups over native DBMSs and 5× over LLM-based systems, demonstrating robust performance improvements.

PVLDB Reference Format:
Saeed Fathollahzadeh, Essam Mansour, and Matthias Boehm. ReSequel: Robust LLM-assisted Query Rewriting and Optimization. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://anonymous.4open.science/r/ReSequel-SIGMOD2026.

1 INTRODUCTION

QUEL (in Ingres [26], based on tuple calculus) and SEQUEL [10, 11] (in System R [4], based on relational algebra) were competing early relational query languages. Ultimately, SEQUEL first became the de-facto standard and later evolved to the SQL standard. The success of SQL and the relational model—despite criticism on specific aspects [8, 45]—was largely due to its (1) declarativity (what not how), (2) flexibility (compose arbitrary complex queries), (3) freedom for automatic optimization, and (4) physical data independence. Despite these compelling properties, all DBMS remain stubbornly sensitive to suboptimal query formulations because query optimization is "a never-ending quest for an increasingly better model and repertoire of optimization and execution techniques" [57], various shortcomings and simplifying assumptions (e.g., the independence assumption and its implication for redundant, correlated predicates) [39], and brittle rewrites (e.g., for unnesting sub-queries) [44].

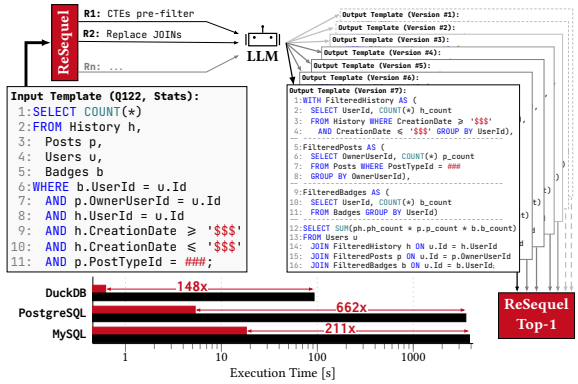


Figure 1: Comparison of ReSequel's Rewriting Impact on Host DBMS Performance. ReSequel rewrites masked templates into multiple variants using LLMs, and then locally verifies them, and selects the Top-1 reconstructed query.

Rule-based Query Rewriting: Many commercial or open-source DBMS have dedicated rule-based query rewriting systems, which transform queries into equivalent, but more efficient forms. These traditional rule engines date back to the early 1990s [48, 49], contain 1,000s of rules, and are applied heuristically before the cost-based optimization of join orders and physical operators. Example rules are the elimination of `DISTINCT` duplicate elimination if the result attributes contain an attribute with unique constraint, query unnesting, and the propagation of constants over joins:

$$R \bowtie_{R.a=S.b} \sigma_{S.b>7}(S) \rightarrow \sigma_{R.a>7}(R) \bowtie_{R.a=S.b} \sigma_{S.b>7}(S) \quad (1)$$

which enables prefiltering both join sides. Such rules are manually crafted by experts over decades [7] or automatically discovered by specialized tools [5, 56]. Since the rules are expressed as source-target equivalences of small patterns, whole-query rewriting requires the application of rules until a fixpoint or number of iterations. The interaction of rules and their application order renders development and debugging difficult and brittle, requiring considerable time, effort, and expertise. Also, the reliance on predefined rules makes rule engines inherently incomplete, potentially missing valuable rewrite opportunities for new queries [16].

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX